

A Reflective Journey: Navigating My Cumulative Experience at Iowa State University

Looking back on my time at Iowa State, the most useful thing I learned was not any single subject. It was a different understanding of what engineering actually asks of a person. I came in expecting the difficulty to live in math, and assumed that once I could solve the equations, everything else would follow. Several years of coursework, team projects, and two internships later, I know the equations were the easiest part. The harder work was learning to take a situation nobody had defined for me, turn it into a problem I could clearly state, and explain my reasoning to people who did not share my background. What follows is not a directory of what I built, but rather a demonstration of how this place changed the way I think.

The first real challenge to my assumptions came in my computer organization and assembly-level programming course, CPRE 3810, where I spent a semester building a processor from the ground up. I expected the difficulty to be the implementation. Instead I spent days convinced my arithmetic was faulty, revising parts that were working correctly. The actual problem was that instructions were using values that had not been produced yet, and I could not see it because I was looking for a mistake rather than for a decision I had never made. Once I understood it as a gap in my design instead of a defect in my code, the work became a series of deliberate choices. What I took from that semester was not a technique. It was the recognition that when I feel stuck, I have usually misunderstood the problem, and that stepping back costs less than pushing forward inside the wrong frame.

My embedded systems course, CPRE 2880, asked for a different kind of humility. I was programming a robot to map its surroundings using several sensors, none of which were reliable. One failed on dark surfaces, another scattered off angled walls, and the rest reported obstacles only after the robot had already found them. My instinct was to identify the most trustworthy sensor and build around it. No such sensor existed. Progress came only when I stopped looking for a dependable input and started designing around the fact that each one failed in a predictable way. Watching an accurate map emerge from measurements I knew to be individually unreliable taught me a principle I have used constantly since. Strong systems are rarely assembled from

perfect parts. They are assembled from imperfect parts arranged so their weaknesses do not overlap.

Those projects also made clear that very few of the answers I needed were in a lecture. Low level work sent me into hundreds of pages of manufacturer documentation, because that information exists nowhere else. When my circuits in electronic circuits and systems, EE 2300, behaved differently in practice than on paper, the explanation came from simulation tools and published design references rather than from the lab handout. For software I learned to go to official documentation and source repositories first, since an answer that worked for a stranger usually hides the assumption that breaks it for me. Over time, I stopped treating outside material as a last resort and started treating it as the normal first step.

The most valuable resource, though, turned out to be other people. Developing a full-stack application in my software development course, COMS 3090, showed me that the hardest problems on a team are rarely technical. They are a teammate who has not contributed to the project, or a decision nobody recorded that everyone now remembers differently. I learned to document choices, keep progress visible, and say we were behind rather than hope we could recover without mentioning it. Office hours, teaching assistants, and classmates who had already hit the same wall saved me more time than any tool did. Asking early stopped feeling like we were incompetent once I saw what it did for the pace of everyone around me.

Much of my growth happened outside of any classroom. Career fairs taught me to explain my work to someone who had ninety seconds and no context, which is a genuinely different skill from explaining it to a professor. Two internships at Motorola Mobility LLC in Chicago expanded my decision making skills exponentially. I contributed to software used by people I will never meet, which changed how I understood my own choices. A defect I failed to catch would eventually reach a customer, and a tool I built would go unused if the people who needed it could not understand it without help. I also learned that technical merit alone does not decide what gets built. Work that cannot be explained to the people responsible for the decision does not move forward, no matter how perfect it is.

Delivering software to real users also made the reach of a design decision impossible to overlook. Technical choices carried economic weight, since performance and reliability translated directly into business outcomes. They carried global weight, because software distributed across regions behaved differently than it does where it was written. They carried social weight, as features handling personal information depend on users understanding what they agreed to. Working across both embedded and cloud systems made the environmental dimension equally clear, since wasted power shortens the life of a device at one end and consumes real resources at the other. I entered this program treating engineering decisions as technical ones. Now, I am convinced they are economic, social, and environmental decisions expressed in technical terms.

An honest account of these years has to include the semesters that went poorly. Several of my more demanding courses, among them data structures (COMS 228), computer organization and assembly-level programming (CPRE 3810), and electronic circuits and systems (EE 2300), exposed a gap between how I was learning and how I was being measured. My project work was consistently strong while my exam performance lagged behind it. What made those semesters valuable was diagnosing the cause rather than accepting the result. I had been learning by building, which serves me well in a lab and fails when an exam asks for recall without context. I adjusted how I prepared, and I learned to read a disappointing outcome as information about my method rather than a judgment of my ability. If I could start over, that is the change I would make the earliest. I would treat how I study as something to be designed rather than something to be endured.

That shift in strategy is exactly what carried into my professional work. This year I had to learn cloud data modeling with no course behind me and a deadline in front of me. My approach was different from the one I would have used as a sophomore. I read the official documentation before touching anything, built a deliberately small version to confirm I understood the model, asked an experienced engineer to review my reasoning before I scaled it, and only then wrote the real implementation. That process was slower on the first day and much faster by the third. It worked because I had learned to verify my understanding early rather than discover it was wrong late.

The most valuable thing Iowa State gave me was the ability to learn on my own. Most of what I used professionally this year came after the relevant course had already ended, and I picked it up quickly because this program had taught me how to approach something unfamiliar. My final year continues that preparation through a senior design project I have to define myself and coursework on how real engineering decisions get made. Going forward, the skill I most want to develop is judgment at a larger scale, meaning the ability to weigh tradeoffs across a whole system rather than one piece of it. Specific tools will come and go throughout my career. The way this university taught me to learn them is what I will keep.